speckit.converge

User Input

```
$ARGUMENTS
```

You **MUST** consider the user input before proceeding (if not empty).

Pre-Execution Checks

Check for extension hooks (before convergence):

- Check if `.specify/extensions.yml` exists in the project root.
- If it exists, read it and look for entries under the `hooks.before_converge` key
- If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally
- Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default.
- For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions:
  - If the hook has no `condition` field, or it is null/empty, treat the hook as executable
  - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation
- For each executable hook, output the following based on its `optional` flag:

  - **Optional hook** (`optional: true`):

    ```
    ## Extension Hooks

    **Optional Pre-Hook**: {extension}
    Command: `/{command}`
    Description: {description}

    Prompt: {prompt}
    To execute: `/{command}`
    ```

  - **Mandatory hook** (`optional: false`):

```
## Extension Hooks

**Automatic Pre-Hook**: {extension}
Executing: `/{command}`
EXECUTE_COMMAND: {command}

Wait for the result of the hook command before proceeding
to the Goal.
```

After emitting the block above you MUST actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook.

- If no hooks are registered or `.specify/extensions.yml` does not exist, skip silently

Goal

Close the gap between what a feature's specification, plan, and tasks call for and what the codebase currently implements. Read `spec.md`, `plan.md`, and `tasks.md` as the **sole source of intent** (with the constitution as governing constraints), assess the current state of the code, determine which requirements, acceptance criteria, plan decisions, and existing tasks are unmet, incomplete, or only partially satisfied, and **append each piece of remaining work as a new, traceable task** at the bottom of `tasks.md` so that `/speckit.implement` can complete it. This command MUST run only after `/speckit.implement` has run on the current `tasks.md`, and after `/speckit.tasks` has produced a complete `tasks.md`.

This is **not** a diff tool and does **not** track changes. It assesses the present state of the code relative to the feature's artifacts — no git, no branch comparison, no history.

Operating Constraints

APPEND-ONLY, NEVER REWRITE: The command's **only** write is appending a new `## Phase N: Convergence` section to `tasks.md`. It MUST NOT:

- modify `spec.md` or `plan.md` in any way;
- rewrite, renumber, reorder, or delete any existing task (including tasks from a prior Convergence phase);
- modify, create, or delete any application code — completing the appended tasks is the job of `/speckit.implement`.

When the codebase already satisfies everything, the command MUST leave `tasks.md` **byte-for-byte unchanged** (no empty Convergence header) and report a clean result.

Constitution Authority: The project constitution (`.specify/memory/constitution.md`) is **non-negotiable**. Code that violates a MUST principle is the highest-severity finding and produces a corresponding remediation task. If the constitution is an unfilled template, skip constitution checks gracefully rather than failing.

Execution Steps

1. Initialize Convergence Context

Run `.specify/scripts/bash/check-prerequisites.sh --json --require-tasks --include-tasks` once from repo root and parse JSON for FEATURE_DIR and AVAILABLE_DOCS. Derive absolute paths:

- SPEC = FEATURE_DIR/spec.md
- PLAN = FEATURE_DIR/plan.md
- TASKS = FEATURE_DIR/tasks.md
- CONSTITUTION = `.specify/memory/constitution.md` (if present)
  If `spec.md`, `plan.md`, or `tasks.md` is missing, STOP with a clear, actionable message naming the prerequisite command to run (`/speckit.specify` for a missing spec, `/speckit.plan` for a missing plan, `/speckit.tasks` for missing tasks). Do not produce partial output. For single quotes in args like “I’m Groot”, use escape syntax: e.g ‘I’'‘m Groot’ (or double-quote if possible: “I’m Groot”).

2. Load Artifacts (Progressive Disclosure)

Load only the minimal necessary context from each artifact:

From spec.md:

- Functional Requirements (FR-###)
- Success Criteria (SC-###) — include only items requiring buildable work; exclude post-launch outcome metrics and business KPIs
- User Stories and their Acceptance Scenarios
- Edge Cases (if present)

From plan.md:

- Architecture/stack choices and technical decisions
- Data Model references
- Phases and named touch-points (files/components the plan says will be created or edited)

- Technical constraints

From tasks.md:

- Task IDs (to compute the next ID and next phase number)
- Descriptions, phase grouping, and referenced file paths

From constitution (if not an unfilled template):

- Principle names and MUST/SHOULD normative statements

3. Build the Intent Inventory

Create an internal model (do not echo raw artifacts):

- **Requirements inventory**: one stable key per FR-### / SC-### / user-story acceptance scenario (e.g. `US1/AC2`), plus the plan decisions and constitution principles that impose buildable obligations.
- **Code-scope map**: from the file paths named in `plan.md` and `tasks.md`, plus a keyword search for the concepts each requirement describes, derive the set of source files and components in scope for assessment. Bound the assessment to these — do **not** infer scope beyond what the artifacts define.

4. Assess the Codebase and Classify Findings

For each item in the intent inventory, inspect the current code in scope and produce a `Finding` only where there is a gap. Classify every finding by **gap type**:

- **`missing`**: the required work is absent from the code entirely.
- **`partial`**: the work exists but does not yet fully satisfy the requirement / acceptance criterion / plan decision.
- **`contradicts`**: the code does something that conflicts with stated intent or a constitution MUST principle.
- **`unrequested`**: the code contains work not called for by the spec, plan, or tasks (surfaced for awareness — converge does **not** delete code, it only appends a task to review/justify or remove it).

Each `Finding` records: a stable id, the `source-ref` it traces to, the `gap-type`, a severity, and a short human-readable description with the evidence (the file/area observed).

Edge cases:

- **Little or no code yet**: treat the entire specified scope as `missing` remaining work rather than failing.
- **Nothing remains**: produce zero findings and follow the converged branch in Step 7.

5. Assign Severity

- **CRITICAL**: violates a constitution MUST principle, or a `missing`/`contradicts` gap that blocks baseline functionality of a P1 user story.
- **HIGH**: a `missing` or `partial` gap on a core functional requirement or acceptance criterion.
- **MEDIUM**: a `partial` gap on a secondary requirement, or an `unrequested` addition with unclear justification.
- **LOW**: minor partial gaps, polish, or low-risk `unrequested` additions.

6. Present the In-Session Findings Summary

Before appending anything, output a compact, severity-graded summary (no file writes yet):

Convergence Findings

ID	Gap Type	Severity	Source	Evidence	Remaining Work
F1	missing	HIGH	FR-008	Example: no append-only guard detected in path/to/module.py when writing tasks.md	Add append-only enforcement

Summary metrics:

- Requirements / acceptance criteria checked
- Plan decisions checked
- Constitution principles checked (or “skipped — template”)
- Findings by gap type (missing / partial / contradicts / unrequested)
- Findings by severity

7. Append Convergence Tasks (or report converged)

If there are one or more actionable findings (`tasks_appended` outcome):

Append to the **end** of `tasks.md`, per the append contract:

1. Scan all existing task IDs; let `M` be the maximum. Determine the next phase number `N` (highest existing phase + 1).

2. Write a single new section header `## Phase N: Convergence`.

3. Emit one checklist item per actionable finding, ordered CRITICAL/HIGH first, assigning zero-padded IDs `T{M+1:03d}`, `T{M+2:03d}`, …:

   ```
   - [ ] T042 <imperative description> per <source-ref> (<gap-
         type>)
   ```

   `<source-ref>` traces the task to its origin: e.g. `FR-003`, `SC-002`, `US1/AC2`, `plan: storage decision`, `Constitution II`.

   `<gap-type>` is one of `missing`, `partial`, `contradicts`, `unrequested`.

   Constitution-violation tasks MUST be emitted first and described as `CRITICAL`.

4. Never reuse or renumber existing IDs. If a prior Convergence phase exists, add a new, separately-numbered one below it — do not touch the old one.

If there are no actionable findings (`converged` outcome):

- Do **not** modify `tasks.md` at all — no empty phase header.
- Report: **“ Converged — the implementation satisfies the spec, plan, and tasks.”**
- Include the summary counts of what was checked.

8. Provide Next Actions (Handoff)

- On `tasks_appended`: state how many tasks were appended under which phase, and recommend running `/speckit.implement` to complete them; note that a follow-up converge run will find fewer or no remaining items.
- On `converged`: recommend proceeding to review / opening a PR. No further implement pass is needed for this feature’s specified scope.

9. Check for extension hooks

After producing the result, check if `.specify/extensions.yml` exists in the project root.

- If it exists, read it and look for entries under the `hooks.after_converge` key
- If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally

- Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default.
- For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions:
  - If the hook has no `condition` field, or it is null/empty, treat the hook as executable
  - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation
- Report the convergence outcome (`converged` or `tasks_appended`) in-session before listing any hooks, so users can decide whether to run optional follow-up commands.
- For each executable hook, output the following based on its `optional` flag:

  - **Optional hook** (`optional: true`):

    ```
    ## Extension Hooks

    **Optional Hook**: {extension}
    Command: `/{command}`
    Description: {description}

    Prompt: {prompt}
    To execute: `/{command}`
    ```

  - **Mandatory hook** (`optional: false`):

    ```
    ## Extension Hooks

    **Automatic Hook**: {extension}
    Executing: `/{command}`
    EXECUTE_COMMAND: {command}
    ```

    After emitting the block above you MUST actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook.

- If no hooks are registered or `.specify/extensions.yml` does not exist, skip silently